InfoBridge: Sprint 2 Status & Hack 1 Plan

Team: The North Remembers · **Project:** DSI 9A (Hyderabad Runners) · **Last updated:** 8 Oct 2026
Hack 1: Saturday 10 October (half-day)

TL;DR

- **Done (PoC v0.1):** the pipeline runs end to end. It reads 5 source formats, retrieves evidence, answers with checked citations, and scores itself against 3 synthetic test cases.
- **Main finding:** plain RAG can't connect "*Ravi Sharma*", "*R. Sharma*" and "*Ravi S.*". Spreadsheet rows hardly get retrieved: Ravi's own membership row ranks 20th of 29. Because of this, the LLM never sees both sides of a conflict.
- **Hack 1 goal:** build the 4 stubbed components (extractor, entity resolver, conflict detector, knowledge graph) and show a **before/after metrics table** on the same test cases.
- **Before Hack 1:** add a Gemini key, run the baseline eval, push to GitHub (see §5).

---

1. What's implemented (Sprint 2)

#	Component	File	Status	What it does
–	Data contracts	`src/infobridge/models.py`	Done	Typed objects every component passes around: `Document`, `Chunk`, `Fact`, `Mention`, `Entity`, `Conflict`, `Citation`, `Claim`, `Answer`
1	Document Ingestion	`ingestion.py`	Done	Turns CSV, Excel, PDF, WhatsApp and email files into small chunks you can cite (one row / message / paragraph each)
2	Information Extractor	`extraction.py`	Stub	Interface and plan only
3	Entity Resolver	`resolution.py`	Stub	Interface and plan only (the scoring function is ready)
4	Knowledge Graph	`graph.py`	Stub	Interface and plan only
5	Vector store	`retrieval.py`	Done	ChromaDB + all-MiniLM-L6-v2 (ONNX), cosine top-k
5b	Conflict Detector	`conflicts.py`	Stub	Interface and plan only (the scoring function is ready)
6	Synthesis Engine	`synthesis.py`	Baseline	Gemini returns JSON claims with `[S#]` citations; each citation is checked against what was retrieved; falls back to "here's where to look" when evidence is weak
7	Provenance Tracker	`provenance.py`	Partial	Shows the trail document → chunk → retrieval rank/score → claim. Extraction and resolution steps come in Hack 1
–	Pipeline	`pipeline.py`	Done	Connects the components. `# HACK 1:` comments mark where new parts go
–	Evaluation	`evaluation.py`	Done	9 metrics against `gold.json`; writes `results/<tag>.json/.md`
–	CLI	`__main__.py`	Done	`ask`, `trail`, `eval` commands

#	Component	File	Status	What it does
–	Tests	`tests/`	15 passing	Ingestion, citation checking, scoring, and a check that every chunk ID in the gold files exists

How chunks are cited

Each chunk ID is `{file}#{locator}`, so every citation points to one exact spot:

Format	One chunk =	Example ID
CSV / Excel	one row (header = row 1, same as Google Sheets)	`members.csv#row=5`
WhatsApp export	one message (multi-line messages joined)	`whatsapp_coach_group.txt#msg=3`
PDF	one paragraph/block	`programme_rules.pdf#page=1&para=2`
Email / text	one paragraph (headers kept as metadata)	`email_long_run_route.txt#para=3`

Synthetic test data (`data/synthetic/`)

Case	Sources	Question	Traps
`case_01_simple`	events.csv, email	Details of the 25 Oct Monthly Long Run	None. This is the easy case
`case_02_medium`	roster.xlsx, WhatsApp, first-aid PDF	Who's at Aid Station 3 and are they first-aid trained?	*Srinivas Rao Kumar / Srini / S. R. Kumar*; *Farhan bhai*; decoy **Srinivas Reddy**; Farhan is untrained
`case_03_conflict`	members.csv, registrations.csv, WhatsApp, rules PDF	Is Ravi Sharma eligible for HMTP 2026B?	*Ravi Sharma / R. Sharma / Ravi S.*; decoy **Ravi Shankar** (lapsed, late); **C1** clearance date 15 Mar vs 22 Jun; **C2** batch start 1 Nov vs 8 Nov. Expected verdict: **YES**

Each case has a `gold.json` listing the facts it contains (with their chunk IDs), every name variant for each person, the planted conflicts, and the expected verdict.

---

2. How to run

```
cd code
uv venv --python 3.10 .venv && uv pip install --python .venv/bin/python -e ".[dev]"
cp .env.example .env            # add GEMINI_API_KEY
source .venv/bin/activate

python -m infobridge ask   data/synthetic/case_03_conflict/sources "Is Ravi Sharma eligible for
the Half Marathon Training Programme Batch 2026B?"
python -m infobridge trail data/synthetic/case_03_conflict/sources "Is Ravi Sharma eligible ...?"
# evidence trail as JSON
python -m infobridge eval --backend gemini --tag baseline_v0.1
# scores all cases
pytest
```

No key? Add `--backend extractive` to any command. It skips the LLM and returns the top retrieved chunks.

---

3. Baseline results so far

Retrieval-only (`results/retrieval_only_v0.1.md`):

Case	Retrieval recall	Fact recall	Citation validity	Conflict recall	Verdict
1 – Simple	1.00	1.00	1.00	–	–
2 – Medium	0.67	0.50	1.00	–	–
3 – Conflict	0.57	0.14	1.00	0.00	0.00

Gemini RAG baseline: not run yet (needs a key; see §5).

What didn't work, and the Hack 1 fix for each

Problem	Evidence	Hack 1 fix
Spreadsheet rows barely get retrieved	All 11 CSV rows in case 3 rank 15th–28th of 29; Ravi's membership row is 20th, his registration row 24th	Look up chunks by resolved entity through the KG, and add BM25 keyword search
The decoy ranks above the real person	Ravi **Shankar**'s lapsed-membership message ranks #4; his member row (#15) beats Ravi Sharma's (#20)	Entity resolution with phone/email as strong keys; filter evidence to the resolved person
Conflicts can't be seen	Dr. Rao's 22 June message is retrieved, the sheet's 15 March isn't, so the LLM sees only one value	A rule-based conflict detector that works on *all* extracted facts, not just the retrieved ones
No structured conflict output	`Answer.conflicts` is always empty	Fill it from the detector

---

4. Hack 1 plan

Goal

The same question, "*Is Ravi Sharma eligible…?*", should now get:

- verdict **YES**
- every rule backed by a correct citation
- **both conflicts flagged**, with both sources for each
- **Ravi Shankar never mentioned**

Then show the before/after metrics table.

Targets

Metric	Baseline	Target
Retrieval recall (case 3)	0.57	≥ 0.90
Structured conflict recall	0.00	≥ 0.90 (C1 + C2 found)
Conflict precision	n/a	≥ 0.80 (nothing flagged in cases 1–2)
Entity resolution F1	n/a	≥ 0.85
Citation validity	1.00	stays at 1.00
Verdict accuracy (case 3)	TBD	1.00

Work split

Four parallel tasks. They only depend on each other through the types in `models.py`. Fill in names:

Task	Owner	File	Depends on
A. Extractor	___	`extraction.py`	– (start first)
B. Entity Resolver	___	`resolution.py`	A's `Fact` output (use hand-written fake facts until A is ready)
C. Conflict Detector	___	`conflicts.py`	A + B (use fake entities/facts until ready)
D. KG + retrieval + integration	___	`graph.py`, `retrieval.py`, `pipeline.py`, `synthesis.py`	B, C

> **Tip:** B and C should each start by writing their unit test with hand-made `Fact` and `Entity` objects from case 3. Then nobody waits on anyone else.

Task A: Information Extractor (`extraction.py`)

Input: `list[Chunk]` → **Output:** `list[Fact]`

1. **Spreadsheet rows: map columns to attributes directly.** No ML needed; confidence 1.0, `extractor="csv_schema"`. Each row's values are already in `chunk.metadata["row"]`.

Sheet	Entity column	Columns → attributes
members.csv	`name`	phone, email, member_id, membership_status, membership_valid_until, medical_clearance_date
registrations.csv	`participant_name`	phone, programme, batch_start → `batch_start_date`, category, registered_on, payment_status
volunteer_roster.xlsx	`full_name`	phone, assigned_station, shift, role
events.csv	`event_name` + `date`	start_time, meeting_point, distance_km, lead_volunteer

   Note: `batch_start` is a fact about the *programme* (`HMTP 2026B`), not the person. Emit it with `entity_mention="HMTP 2026B"`.

2. **WhatsApp, PDF and email text: use Gemini to extract JSON.** Use a fixed list of allowed attribute names (the same names as above, plus `event_result`, `training_attendance`, `first_aid_certified`, `registration_deadline`). Prompt: "*Extract (entity, attribute, value) facts from this message. Only use these attributes. Return* `[]` *if none.*" Send one LLM call per document (all its chunks together) to stay under rate limits.

3. **Normalise values** so they can be compared:
   - dates → `YYYY-MM-DD` (`"22 June 2026"`, `"15/03/2026"` → ISO; WhatsApp dates are dd/mm)
   - phone numbers → last 10 digits
   - times → `HH:MM`
   - Always keep the original text in `raw_value`.

4. **Done when:** case 3 produces facts for Ravi's membership, registration, clearance (both values), 10K result and attendance, plus the two batch start values.

Task B: Entity Resolver (`resolution.py`)

Input: `list[Fact]` → **Output:** `list[Entity]` (each mention carries its chunk ID; record *why* things were merged in `match_evidence`)

Merge rules, in this order:

1. **Strong keys.** Same normalised phone or email → same person. This links `R. Sharma` (9848022314) to `Ravi Sharma` (+91 98480 22314).
2. **Compatible names.**
   - Initials: `R. Sharma` ~ `Ravi Sharma`, `S. R. Kumar` ~ `Srinivas Rao Kumar`, `A. Iyer` ~ `Ananya Iyer`.
   - Nickname prefixes: `Srini` ~ `Srinivas`.
   - Truncated surnames: `Ravi S.` ~ `Ravi Sharma`.
   - Honorifics stripped: `Farhan bhai` → `Farhan`; `Dr.`, `Coach`.
   - Fuzzy matching: `rapidfuzz.fuzz.token_sort_ratio` ≥ 90.
3. **Never merge** when two full surnames differ (`Sharma` ≠ `Shankar`, `Reddy` ≠ `Kumar`), even if first names match.
4. **Ambiguous cases** (`Ravi S.` could be Sharma or Shankar): use context. Take the candidate who shares the most attributes or co-occurring entities; if it's a tie, keep them separate and lower the confidence.

Done when: `entity_resolution_f1(entities, gold)` ≥ 0.85 on all cases, and Ravi Shankar and Srinivas Reddy stay separate. *(Get mention text from `Fact.entity_mention`. The gold file also lists sender names like "Srini" as mentions, so A should emit a fact for the WhatsApp sender too.)*

Task C: Conflict Detector (`conflicts.py`)

Input: `list[Entity]`, `list[Fact]` → **Output:** `list[Conflict]`

1. Map each fact to its entity: `chunk_id` + `entity_mention` → `entity_id`.
2. Group the facts by `(entity_id, attribute)`.
3. If a group has more than one distinct **normalised** value, that's a `Conflict`, with one `ConflictValue` per value and the chunk it came from.
4. Ignore attributes that can legitimately differ: `phone` formatting (it's normalised already), and things that are expected to have several values.
5. Severity:
   - **high**: the conflict changes whether a rule passes
   - **medium**: dates or counts
   - **low**: anything else
6. Suggestion: "*prefer the most recent source*", or "*prefer the official document over chat*" (e.g. the rules PDF over the registration sheet for C2).

Done when: case 3 → exactly C1 and C2; cases 1–2 → no conflicts.

Task D: KG, retrieval & integration

1. `graph.py` using a NetworkX `MultiDiGraph`:
   - entity nodes, plus value nodes or edge attributes
   - edges `entity --attribute--> value`, each storing `chunk_id`, `doc_id`, `fact_id`
   - edges involved in a conflict get a `conflict_id`
   - helper `chunks_for_entity(entity_id) -> set[chunk_id]`
2. **Better retrieval** (`retrieval.py` / `pipeline.ask`):
   1. Find the entities named in the question (match the question text against entity mentions).
   2. Add **all chunks for those entities** from the KG to the dense top-k, plus the chunks for linked programmes or events.
   3. Optional: a BM25 keyword channel (`rank_bm25`) for IDs and names.

    4. Drop chunks about look-alike entities that weren't asked about (Ravi Shankar).
3. **Conflict-aware synthesis** (`synthesis.py`): add a `KNOWN CONFLICTS` block to the prompt, citing both sides with their `[S#]` labels. Tell the model to mention every relevant conflict and whether it changes the verdict.
4. **`pipeline.py`:** connect the parts at the `# HACK 1:` markers. Store `self.facts`, `self.entities`, `self.conflicts`, `self.graph`, and set `answer.conflicts`.
5. **Evaluation:** in `evaluation.run`, also record `entity_resolution_f1(bridge.entities, gold)` and add a **conflict precision** metric (flagged conflicts that match a gold conflict ÷ all flagged).
6. **`provenance.py`:** for each citation, add `extracted_fact` and `resolved_entity (match_evidence)` steps to the trail.

Hack-day timeline (about 4.5 h)

Time	Everyone	A	B	C	D
0:00–0:15	Pull, run `pytest`, agree on attribute names				
0:15–1:45		Table mapping → LLM extraction → normalisation	Tests with fake facts → strong keys → name rules	Tests with fake data → grouping → severity	KG + `chunks_for_entity`; prompt block
1:45–2:00	**Sync:** A hands real facts to B and C				
2:00–3:00		Fix extraction gaps B/C report	Tune on real facts; ER F1	Run on real entities	Connect pipeline + retrieval expansion
3:00–3:30	**Integration:** run `eval --tag hack1_v0.1`, compare with baseline				
3:30–4:15	Fix the worst failure; re-run eval				
4:15–4:30	Record demo; write the issues log				

If we fall behind: use spreadsheet facts, phone-based resolution and the conflict detector, plus a small **regex date extractor** for chat and PDF text (a date near keywords like *clearance* or *begins*). Each conflict has one side in plain text (C1 in a WhatsApp message, C2 in the PDF), so the regex extractor is what still catches them. Leave full LLM extraction for Sprint 3.

Demo script (about 3 min video)

1. **Problem (20 s):** show the 4 messy source files for Ravi Sharma: three name forms, a decoy, two contradictions.
2. **Before (40 s):** run `ask` with the baseline. It misses the membership and registration rows and doesn't mention the conflicts.
3. **After (60 s):** the same question gives YES, cited claims, and 2 conflicts flagged with both sources.
4. **Evidence (30 s):** run `trail` and follow one claim back to `members.csv` row 5.
5. **Numbers (30 s):** show the before/after metrics table.
6. **What doesn't work yet, and our questions for instructors (20 s).**

Hack 1 deliverables checklist

☐ **Working core PoC v0.1** pushed to GitHub (GitHub)

☐ **Technical Design Doc revised** with Hack 1 results, a before/after table, and what works and what doesn't (Moodle)

☐ **Demo video:** input → technology → output (Moodle)

☐ **Specific issues / questions** for instructor feedback (in the doc, §10)

☐ **Feedback log** for Sprint 3 (`docs/hack1_feedback_log.md`: what broke, what took longer, which assumptions were wrong)

---

5. Sprint 2: left to do before Hack 1

☐ Get a Gemini API key (https://aistudio.google.com/apikey) → `.env`

☐ `python -m infobridge eval --backend gemini --tag baseline_v0.1`

☐ Paste `results/baseline_v0.1.md` into §6.2 of `technical_design_doc.md`, plus 2–3 lines on what Gemini did on case 3

☐ Create the team GitHub repo and push (`git remote add origin … && git push -u origin main`)

☐ Submit the Technical Design Doc on Moodle (convert to PDF/LaTeX if needed)

☐ Everyone clones the repo, runs `pytest`, and installs Hack 1 libraries: `uv pip install --python .venv/bin/python -e ".[hack1]"`

☐ Assign owners A–D above

---

6. Risks

Risk	Mitigation
Gemini free-tier rate limits during the hack	One extraction call per document, not per chunk; cache LLM outputs to `results/cache/`; `--backend extractive` for quick runs
LLM extraction returns inconsistent attribute names	A fixed list of allowed attributes in the prompt; drop anything not on it
`Ravi S.` is ambiguous (Sharma or Shankar)	Use context; if it's still a tie, keep them separate rather than merge wrongly
Keyword-based fact scoring misses paraphrases	Spot-check answers by hand; add an LLM judge in Sprint 3
Overfitting to our own 3 cases	Sprint 3: add 2–3 new cases written by a teammate who didn't build the resolver

7. After Hack 1 (Sprint 3 preview)

- More synthetic cases: harder conflicts, 5+ sources, cases where the information simply isn't there
- One real open-data scenario (OGD / WHO / OpenAlex) to show it works beyond our own data
- An LLM-judge for fact recall; a human rubric for coherence
- Conflict-resolution suggestions, evaluated
- Latency and caching